

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE GOIÁS

Escola Politécnica e de Artes — Curso de Análise e Desenvolvimento de Sistemas

ATIVIDADE PRÁTICA

Arquiteturas de Software com Java

Sistema de Gerenciamento de Biblioteca

Disciplina	Análise e Desenvolvimento de Sistemas
Modalidade	Individual
Valor	10,0 pontos
Entrega	Conforme definido no Moodle — repositório Git público (GitHub/GitLab)

1. Contexto

Uma rede de bibliotecas municipais necessita de um sistema para gerenciar o acervo de livros e o fluxo de empréstimos. Atualmente, os processos são realizados manualmente em planilhas, o que gera inconsistências nos registros e dificulta o controle de devoluções em atraso. A proposta é desenvolver uma aplicação Java que modele os principais casos de uso do sistema, com ênfase na organização arquitetural do código.

A atividade é estruturada em três etapas progressivas, cada uma correspondendo a um estilo arquitetural distinto. O estudante deve implementar o sistema de forma incremental, evoluindo da Arquitetura em Camadas para a Arquitetura Hexagonal e, por fim, introduzindo comunicação assíncrona por eventos.

Entidades do Domínio

O domínio central do sistema é composto pelas seguintes entidades:

- **Livro**: identificador, título, autor, ISBN, quantidade disponível.
 - Atributos: `Long id`, `String titulo`, `String autor`, `String isbn`, `int quantidadeDisponivel`
- **Usuário**: identificador, nome, e-mail, situação (ATIVO / SUSPENSO).
- **Empréstimo**: identificador, livro, usuário, data de retirada, data prevista de devolução, situação (ATIVO / DEVOLVIDO / ATRASADO).

2. Etapa 1 — Arquitetura em Camadas (3,0 pontos)

2.1 Descrição

Nesta etapa, o sistema deve ser implementado seguindo a Arquitetura em Camadas, com separação clara entre as camadas de Domínio, Aplicação, Infraestrutura e Apresentação. Nenhuma camada de nível inferior deve ser referenciada por camadas de nível superior diretamente — a dependência deve fluir de cima para baixo.

2.2 Requisitos de Implementação

A implementação deve contemplar os seguintes elementos:

a) Camada de Domínio

- Implementar as classes `Livro`, `Usuario` e `Emprestimo` como POJOs (Plain Old Java Objects), sem dependências de frameworks.
- Implementar regras de negócio dentro das entidades: o método `realizarEmprestimo()` em `Livro` deve verificar disponibilidade antes de decrementar o estoque.
- Implementar o enum `SituacaoEmprestimo` (ATIVO, DEVOLVIDO, ATRASADO) e `SituacaoUsuario` (ATIVO, SUSPENSO).

b) Camada de Infraestrutura

- Implementar repositórios em memória (`HashMap`) para `LivroRepositorio`, `UsuarioRepositorio` e `EmprestimoRepositorio`.
- Cada repositório deve oferecer os métodos: `salvar()`, `buscarPorId()`, `listarTodos()` e `remover()`.

c) Camada de Aplicação

Implementar os seguintes casos de uso como métodos da classe de serviço correspondente:

```
// EmprestimoServico - casos de uso obrigatórios
realizarEmprestimo(Long usuarioId, Long livroId) → Emprestimo
registrarDevolucao(Long emprestimoId)           → void
listarEmprestimosAtivos()                       → List<Emprestimo>
verificarAtrasos()                             → List<Emprestimo> // compara
datas
```

d) Camada de Apresentação

- Implementar uma classe `Main` que demonstre, via saída no console, a execução de pelo menos um fluxo completo: cadastro de livro → cadastro de usuário → realização de empréstimo → devolução.

2.3 Estrutura de Pacotes Esperada

```
biblioteca/
  dominio/
    Livro.java
    Usuario.java
    Emprestimo.java
    SituacaoEmprestimo.java
    SituacaoUsuario.java
  infraestrutura/
```

```
LivroRepositorio.java
UsuarioRepositorio.java
EmprestimoRepositorio.java
aplicacao/
  LivroServico.java
  UsuarioServico.java
  EmprestimoServico.java
apresentacao/
  Main.java
```

Restrição importante

As classes da camada de Domínio não podem importar nenhuma classe das camadas de Infraestrutura ou Aplicação. Qualquer violação dessa regra acarreta desconto direto na avaliação desta etapa.

3. Etapa 2 — Arquitetura Hexagonal (4,0 pontos)

3.1 Descrição

Com base no código da Etapa 1, esta etapa exige a refatoração do sistema para o modelo de Ports and Adapters (Arquitetura Hexagonal). O objetivo é isolar completamente o domínio de qualquer detalhe de infraestrutura por meio de interfaces (portas), implementadas por adaptadores concretos.

Ao final desta etapa, deve ser possível substituir qualquer adaptador — por exemplo, trocar o repositório em memória por um baseado em arquivo CSV — sem modificar uma única linha de código do domínio ou dos serviços de aplicação.

3.2 Requisitos de Implementação

a) Portas de Saída (Output Ports)

Converter os repositórios concretos em interfaces, definidas no pacote de domínio:

```
// porta/saida/PortaLivroRepositorio.java
public interface PortaLivroRepositorio {
    void salvar(Livro livro);
    Optional<Livro> buscarPorId(Long id);
    List<Livro> listarTodos();
    void remover(Long id);
}

// Repetir o padrão para PortaUsuarioRepositorio e PortaEmprestimoRepositorio
```

b) Portas de Saída — Notificação

Criar uma porta de saída para envio de notificações ao usuário:

```
// porta/saida/PortaNotificacao.java
public interface PortaNotificacao {
    void notificarAtraso(Usuario usuario, Emprestimo emprestimo);
}
```

c) Adaptadores de Infraestrutura

Implementar dois adaptadores para cada porta de repositório:

- Adaptador em memória (`LivroRepositorioMemoria`) — migrado da Etapa 1.
- Adaptador CSV (`LivroRepositorioCsv`) — persiste os dados em um arquivo `livros.csv` usando a API padrão de I/O do Java.
- Adaptador de notificação por console (`NotificacaoConsole`) — exibe a mensagem no terminal.

d) Porta de Entrada (Input Port)

Definir uma interface de caso de uso como porta de entrada do domínio:

```
// porta/entrada/PortaEmprestimo.java
public interface PortaEmprestimo {
    Emprestimo realizarEmprestimo(Long usuarioId, Long livroId);
    void registrarDevolucao(Long emprestimoId);
    List<Emprestimo> listarEmprestimosAtivos();
    List<Emprestimo> verificarAtrasos();
}

// EmprestimoServico implementa PortaEmprestimo
```

e) Composição e Demonstração

A classe Main deve demonstrar a troca de adaptador em tempo de inicialização, instanciando o serviço de empréstimos com o repositório em memória e, em seguida, com o repositório CSV, sem alterar nenhuma lógica de negócio:

```
// Composição com adaptador em memória
PortaLivroRepositorio livroRepo = new LivroRepositorioMemoria();
PortaNotificacao notif = new NotificacaoConsole();
PortaEmprestimo servico = new EmprestimoServico(livroRepo, ..., notif);

// Mesma lógica — adaptador substituído sem tocar em EmprestimoServico
PortaLivroRepositorio livroRepoCsv = new LivroRepositorioCsv("livros.csv");
PortaEmprestimo servicoCsv = new EmprestimoServico(livroRepoCsv, ..., notif);
```

3.3 Estrutura de Pacotes Esperada

```
biblioteca/
  dominio/
    Livro.java | Usuario.java | Emprestimo.java | Enums...
  porta/
    entrada/
      PortaEmprestimo.java
    saida/
      PortaLivroRepositorio.java
      PortaUsuarioRepositorio.java
      PortaEmprestimoRepositorio.java
      PortaNotificacao.java
  servico/
    EmprestimoServico.java // implementa PortaEmprestimo
  infraestrutura/
    adaptador/
      LivroRepositorioMemoria.java
```

```
LivroRepositorioCsv.java
UsuarioRepositorioMemoria.java
EmprestimoRepositorioMemoria.java
NotificacaoConsole.java
apresentacao/
    Main.java
```

Critério de aceitação da Etapa 2

A implementação será considerada válida somente se nenhuma classe do pacote domínio importar classes dos pacotes infraestrutura ou apresentacao. O núcleo do domínio deve depender exclusivamente de suas próprias abstrações (portas) e de tipos da biblioteca padrão do Java.

4. Etapa 3 — Comunicação Assíncrona por Eventos (3,0 pontos)

4.1 Descrição

Nesta etapa, o sistema deve ser estendido com um mecanismo de publicação e consumo de eventos (padrão Publisher/Subscriber). O objetivo é desacoplar os efeitos colaterais de um caso de uso — como o envio de notificações e a atualização de logs — do fluxo principal da transação, reproduzindo o comportamento de um sistema baseado em mensageria assíncrona.

4.2 Requisitos de Implementação

a) Contrato de Evento

Criar as seguintes classes de evento no pacote domínio/evento:

```
// EmprestimoRealizadoEvento.java
public record EmprestimoRealizadoEvento(
    Long emprestimoId,
    Long usuarioId,
    Long livroId,
    LocalDate dataRetirada
) {}

// DevolucaoRegistradaEvento.java
public record DevolucaoRegistradaEvento(
    Long emprestimoId,
    LocalDate dataDevolucao,
    boolean comAtraso
) {}
```

Obs.: O uso de records (disponível a partir do Java 16) é incentivado por sua imutabilidade natural. Caso o ambiente utilize versão anterior, o padrão pode ser implementado com classes finais convencionais.

b) Barramento de Eventos Genérico

Implementar um EventBus tipado usando generics, capaz de gerenciar assinaturas e publicações de qualquer tipo de evento:

```
public class EventBus<T> {
    private final List<Consumer<T>> assinantes = new ArrayList<>();
```

```

    public void assinar(Consumer<T> handler) {
        assinantes.add(handler);
    }

    public void publicar(T evento) {
        assinantes.forEach(h -> h.accept(evento));
    }
}

```

c) Integração com o Serviço de Empréstimo

Modificar `EmprestimoServico` para publicar eventos ao concluir cada caso de uso. O serviço não deve conhecer os consumidores dos eventos — apenas publicar.

```

// Ao realizar empréstimo:
eventosBusEmprestimo.publicar(new EmprestimoRealizadoEvento(...));

// Ao registrar devolução:
eventosBusDevolucao.publicar(new DevolucaoRegistradaEvento(...));

```

d) Consumidores de Eventos (Handlers)

Implementar ao menos dois consumidores independentes:

- **ServicoDeNotificacao:** consome `EmprestimoRealizadoEvento` e notifica o usuário sobre a data prevista de devolução.
- **ServicoDeLog:** consome ambos os eventos e registra a operação em um arquivo de log (`biblioteca.log`) com timestamp.

e) Demonstração

A classe `Main` deve registrar os consumidores e executar uma sequência completa que prove o funcionamento assíncrono dos eventos — a publicação deve ativar ambos os handlers automaticamente, sem chamada direta.

Ponto de atenção — Desacoplamento

`EmprestimoServico` não pode importar `ServicoDeNotificacao` nem `ServicoDeLog`. A relação entre eles deve existir exclusivamente por meio do `EventBus`. Qualquer dependência direta entre publicador e consumidor descaracteriza o padrão e resulta em desconto.

5. Critérios de Avaliação

A pontuação é distribuída conforme a tabela a seguir. Cada critério é avaliado de forma independente:

Critério	Descrição	Pontos
----------	-----------	--------

Etapa 1 — Separação de camadas	Ausência de dependências cruzadas entre camadas; coesão das classes.	1,5
Etapa 1 — Casos de uso	Implementação correta dos casos de uso com regras de negócio no domínio.	1,5
Etapa 2 — Ports and Adapters	Interfaces no domínio; adaptadores na infraestrutura; serviço usa apenas portas.	2,0
Etapa 2 — Adaptador CSV	Leitura e escrita corretas no arquivo CSV com serialização adequada.	1,0
Etapa 2 — Substituição de adaptador	Demonstração funcional da troca de adaptador sem alterar o domínio.	1,0
Etapa 3 — EventBus genérico	Implementação correta com generics; suporte a múltiplos assinantes.	1,0
Etapa 3 — Handlers desacoplados	Consumidores independentes; ausência de dependência direta entre publicador e handlers.	1,0
Etapa 3 — Log com timestamp	Registro correto em arquivo com data/hora da operação.	1,0
Total		10,0

Penalizações

- **Código que não compila:** zero na etapa correspondente.
- **Ausência de histórico no Git** (único commit com todo o código): desconto de 1,0 ponto.
- **Código gerado por IA sem compreensão demonstrada:** o estudante poderá ser convocado para apresentação oral da solução.

6. Instruções de Entrega

O projeto deve ser entregue como repositório Git público (GitHub ou GitLab) contendo:

1. Código-fonte organizado conforme as estruturas de pacotes indicadas nas Seções 2.3 e 3.3.
2. Arquivo `README.md` descrevendo: (a) como compilar e executar o projeto; (b) decisões de design relevantes; (c) dificuldades encontradas e como foram resolvidas.
3. Histórico de commits organizado, com mensagens descritivas (mínimo de 5 commits ao longo do desenvolvimento).
4. O link do repositório deve ser submetido pelo Moodle até a data definida na plataforma.

Recomendações Técnicas

- **Java 17+** é recomendado pelo suporte a records e outros recursos de linguagem úteis na Etapa 3.
- **Maven ou Gradle** podem ser utilizados para gerenciamento de build, mas não são obrigatórios. A atividade pode ser realizada com projeto Java simples.
- **Frameworks externos** (Spring, Quarkus, etc.) **não são permitidos**. O objetivo é demonstrar os princípios arquiteturais com Java puro.
- **JUnit 5** é permitido e valorizado para testes unitários das regras de domínio.